

Manualul Infrastructurii

merox.dev — homelab Kubernetes + Oracle Cloud VPS

De la principii generale la implementare:
cum funcționează fiecare componentă, de ce a fost aleasă
și cum se operează — la nivel de senior sysadmin / arhitect.

Generat: 12 iunie 2026

Sursa: repo `meroxdotdev/infrastructure`

Talos v1.13.3 · Kubernetes v1.36.1 · Flux v2

Cuprins

Partea I — Fundamente: containere, Kubernetes, GitOps.....	3
1. Cum funcționează un container, de fapt.....	3
2. Arhitectura Kubernetes, piesă cu piesă.....	4
3. GitOps și Flux: clusterul ca funcție de git.....	6
Partea II — Sistemul de operare: Talos Linux.....	8
4. Talos: OS imutabil, gestionat exclusiv prin API.....	8
Partea III — Rețeaua, cap-coadă.....	10
5. Cilium și viața unui pachet în cluster	10
6. DNS: split-horizon, Pi-hole, Unbound, k8s-gateway	11
7. Expunerea spre internet: Cloudflare Tunnel, TLS, cert-manager.....	12
8. Tailscale: planul de management.....	12
9. Traefik: reverse proxy-ul VPS-ului.....	13
Partea IV — Stocare și date	14
10. Longhorn: block storage distribuit	14
11. Garage: S3 self-hosted ca destinație de backup	15
12. NFS și CSI.....	15
Partea V — Identitate și secrete	17
13. Authentik: SSO, OIDC și forward-auth.....	17
14. Secrete: SOPS/age, Ansible Vault, blast radius	18
Partea VI — Observabilitate.....	20
15. Prometheus, Grafana, Alertmanager	20
16. Loguri: Loki și Promtail.....	21
17. Restul flotei: Netdata, Uptime Kuma, Beszel, Dozzle, Glances.....	21
Partea VII — VPS-ul: Ansible, Terraform și serviciile Docker.....	23
18. Ansible și Terraform: VPS-ul ca artefact reproductibil	23
19. Serviciile de pe VPS, unul câte unul.....	24
Partea VIII — Aplicațiile din cluster.....	25
20. Media stack-ul și restul aplicațiilor.....	25
Partea IX — Backup și Disaster Recovery	27
21. Strategia de backup: doar stare reală, off-site reciproc	27
22. Scenarii DR și proceduri de restaurare	29
Partea X — Agenții AI și operarea de zi cu zi.....	30
23. OpenClaw: agenți AI peste infrastructură	30
24. Day-2 operations: comenzi, tipare de debugging, upgrade-uri.....	30
25. Glosar	32

PARTEA I

Fundamente: containere, Kubernetes, GitOps

Toată infrastructura asta se sprijină pe trei idei. Dacă le stăpânești, restul documentului devine o serie de aplicații ale lor:

1. **Izolare prin kernel, nu prin virtualizare** — containerele sunt procese Linux obișnuite, izolate cu mecanisme de kernel. De aici vin și viteza, și limitele lor de securitate.
2. **Reconciliere declarativă** — nu spui sistemului ce să facă, ci cum trebuie să arate. Controllere rulează în buclă infinită comparând starea dorită cu cea reală și corectând diferența. Kubernetes, Flux, cert-manager, Longhorn — toate sunt variațiuni ale acestei bucle.
3. **Git ca sursă unică de adevăr** — dacă starea dorită e descrisă în fișiere, fișierele pot sta în git. Deploy = `git push`. Rollback = `git revert`. Audit = `git log`. Disaster recovery = clonezi repo-ul și lași reconcilierea să-și facă treaba.

1. Cum funcționează un container, de fapt

Un container **nu e o mașină virtuală mică**. Nu există hypervisor, nu există kernel separat. Un container e un proces normal pe host, pornit cu câteva facilități de kernel activate:

1.1 Namespaces — ce vede procesul

Kernelul Linux poate da unui proces o vedere privată asupra unor resurse globale. Fiecare tip de namespace izolează altceva:

Namespace	Izolează	Efect practic
<code>pid</code>	Arborele de procese	Procesul din container se vede pe sine ca PID 1; nu vede procesele hostului.
<code>net</code>	Stiva de rețea	Container = interfețe, rute, iptables proprii. De aici nevoia de CNI/bridge ca să comunice.
<code>mnt</code>	Mount points	Containerul vede propriul filesystem root, nu pe al hostului.
<code>uts</code>	Hostname	Containerul are hostname propriu.
<code>ipc</code> , <code>user</code> , <code>cgroup</code>	IPC, UID mapping, ierarhia cgroup	Memorie partajată separată; opțional root-ul din container $\neq$ root pe host.

1.2 Cgroups — ce consumă procesul

Control groups (v2 în kernel-urile moderne) limitează și contorizează resursele: CPU, memorie, IO, număr de procese. Când pui `resources.limits.memory: 512Mi` pe un pod, kubelet scrie limita în cgroup-ul containerului; dacă procesul o depășește, kernelul îl omoră (OOMKill) — de aceea vezi `OOMKilled` în `kubectl describe pod`. Limita de CPU funcționează prin throttling (CFS quota): procesul nu e omorât, doar încetinit.

1.3 Imagini și OverlayFS — ce conține procesul

O imagine de container e un tar de straturi (layers) plus un manifest JSON. Fiecare instrucțiune `RUN / COPY` din Dockerfile produce un layer = diferența de fișiere față de stratul anterior. La rulare, OverlayFS suprapune straturile read-only și adaugă un strat writable deasupra (copy-on-write). Consecințe practice de care te lovești zilnic:

- Straturile se **partajează** între imagini — 10 imagini bazate pe `debian:bookworm` ocupă baza o singură dată.
- Tot ce scrie containerul în stratul writable **dispare la recreare** — de aceea orice stare persistentă trebuie pe volum (Docker volume, PVC în K8s).
- Un tag (`:latest`, `:1.2`) e un pointer mutabil; **digestul** (`@sha256:...`) e imutabil. Renovate face pin pe versiuni exacte tocmai ca deploy-urile să fie reproductibile.

1.4 Rețeaua Docker pe VPS

Docker creează un bridge (switch virtual L2) per rețea compose. Fiecare container primește o pereche **veth** — un „patch cable” virtual: un capăt în namespace-ul containerului, celălalt în bridge. Ieșirea spre internet se face prin NAT (reguli iptables MASQUERADE scrise de Docker). DNS-ul intern al Docker rezolvă numele containerelor din aceeași rețea — de aceea în compose poți scrie `http://authentik-server:9000`.

ÎN ACEST SETUP

Pe VPS, serviciile Docker stau în rețele cu IP-uri statice `172.25.10.x` (convenție documentată în `vps/BEST_PRACTICES.md`). IP-urile statice contează pentru că Traefik și Pi-hole referențiază servicii prin IP în câteva locuri, iar la un `docker compose up` ordinea de pornire nu garantează aceleași IP-uri dinamice.

1.5 Limita de securitate

Toate containerele de pe un host **împart același kernel**. Un exploit de kernel evadează din orice container. De aceea: (a) un container compromis pe VPS e tratat ca „VPS-ul compromis” în analiza de blast radius (cap. 14); (b) mediile serioase multi-tenant folosesc VM-uri, nu containere, ca graniță de securitate. În homelab accepți riscul, dar îl gestionezi limitând ce secrete stau pe fiecare mașină.

2. Arhitectura Kubernetes, piesă cu piesă

Kubernetes e, în esență, **o bază de date cu controllere**. Totul devine clar dacă reții asta:

CONTROL PLANE (pe fiecare din cele 3 noduri Talos)

```

kube-apiserver ← singura uşă de intrare (REST + watch)
| singurul care vorbeşte cu ▼
etcd (key-value store, consens Raft, quorum 2/3)

kube-scheduler → decide PE CE NOD merge un pod
kube-controller-manager → buclele: Deployment→RS→Pod etc.

```

WORKER (acelaşi nod – control-plane-urile rulează şi apps)

```

kubelet → agent pe nod; porneşte/opreşte containere
containerd → runtime-ul efectiv (prin interfaţa CRI)
cilium → CNI: reţea pod-to-pod + înlocuieşte kube-proxy

```

2.1 apiserver şi etcd: creierul

Tot ce există în cluster — pods, secrets, configmaps, CRD-uri — e un obiect JSON în etcd, accesat exclusiv prin apiserver. Nimeni (nici scheduler-ul, nici kubelet) nu vorbeşte direct cu etcd. Componentele folosesc **watch**: o conexiune long-lived prin care apiserver-ul le împinge fiecare schimbare relevantă. De aceea Kubernetes reacţionează în milisecunde fără polling.

etcd foloseşte Raft pentru consens: fiecare scriere trebuie confirmată de majoritatea membrilor (quorum). Cu 3 noduri, quorum = 2: clusterul supravieţuieşte pierderii unui nod, dar la pierderea a două **etcd devine read-only** şi clusterul îngheaţă (workload-urile rulează în continuare, dar nimic nu se mai poate schimba). De aici regula numărului impar de control-plane-uri: 4 noduri tolerează tot 1 pierdut (quorum 3/4), deci al patrulea nu adaugă nimic decât risc.

ÎN ACEST SETUP

Cele 3 noduri Talos sunt **toate** control-plane şi rulează şi aplicaţii (taint-ul de control-plane e scos). La 3 noduri fizice e alegerea corectă: HA pentru etcd fără să sacrifici maşini doar pentru control plane. API-ul e accesat printr-un **VIP partajat** (10.57.57.88) — vezi cap. 4.

2.2 Lanţul Deployment → ReplicaSet → Pod

Modelul de reconciliere e compoziţional — fiecare controller face un singur pas mic:

- Tu creezi un **Deployment** („vreau 1 replică din jellyfin:10.9, strategie RollingUpdate”).
- Deployment-controller creează un **ReplicaSet** pentru versiunea curentă a template-ului. La schimbarea imaginii, creează un RS nou şi scade gradual pe cel vechi — aşa funcţionează rolling update şi rollback (RS-ul vechi rămâne la scale 0, gata de revenire).
- ReplicaSet-controller creează/şterge **Pods** ca să atingă replica count.
- **Scheduler-ul** vede podul fără `nodeName` şi alege nodul: filtrează (are nodul resursele cerute? tolerează taint-urile? are label-urile din nodeSelector — ex. `gpu.intel.com/i915` pentru Jellyfin?) apoi punctează şi alege.
- **kubelet-ul** de pe nodul ales vede podul prin watch şi cere lui containerd să tragă imaginea şi să pornească containerele.

Alte tipuri de workload pe care le ai în cluster: **StatefulSet** (identitate stabilă per replică + PVC propriu — Prometheus, Loki), **DaemonSet** (un pod pe fiecare nod — Cilium, Promtail, Longhorn manager), **Job/CronJob** (rulări finite).

2.3 Services: IP-uri stabile peste pods efemere

Pods-urile mor și primesc alt IP. Un **Service** e un IP virtual stabil (ClusterIP) + un selector de label-uri; controllerul de endpoints menține lista de pod-IP-uri vii din spatele lui. Implementarea clasică (kube-proxy + iptables) rescrie destinația pachetelor; aici e **înlocuită de Cilium cu eBPF** (cap. 5) — mai rapid și fără mii de reguli iptables. Tipuri:

Tip	Ce face	Folosit aici pentru
ClusterIP	IP virtual accesibil doar din cluster	Aproape tot (app↔app)
LoadBalancer	IP extern; în cloud îl dă providerul, on-prem trebuie să-l anunțe cineva	Cilium L2 anunță 10.57.57.101/.102/.103/.111/.112 pe LAN
NodePort	Port pe fiecare nod	Evitat — IP-uri LB sunt mai curate

2.4 PV, PVC, StorageClass, CSI: contractul de stocare

Stocarea e decuplată în trei obiecte ca aplicația să nu știe nimic despre infrastructura de dedesubt: aplicația cere un **PVC** („vreau 10Gi RWO”), clusterul îl satisface cu un **PV** (volumul concret), iar **StorageClass**-ul spune cine și cum provisionează (aici: `longhorn` pentru block replicat, `nfs` pentru media de pe NAS). **CSI** (Container Storage Interface) e standardul prin care driverele de storage (Longhorn, csi-driver-nfs) se conectează la kubelet — de aceea poți schimba storage-ul fără să atingi Kubernetes-ul în sine. Subtilitate care a mușcat la DR: un PV `Released` ține minte vechiul `claimRef` și refuză re-bind până nu-l cureți — scriptul de restore o face automat (cap. 22).

2.5 CRD-uri și operatori: cum se extinde Kubernetes

Un **CustomResourceDefinition** adaugă un tip nou de obiect în apiserver (ex. `HelmRelease`, `Certificate`, `Volume` al Longhorn). Un **operator** = CRD + controller care reconciliază acel tip. Asta e mecanismul prin care jumătate din stack-ul tău există: Flux, cert-manager, Longhorn, Prometheus Operator, Gateway API — toate sunt operatori care traduc obiecte declarative de nivel înalt în resurse de bază. Când ceva „nu se întâmplă”, primul reflex: `kubectl describe` pe resursa custom și citește `status` și `events` — acolo raportează controllerul de ce refuză.

2.6 Helm: pachete și templating

Helm e managerul de pachete: un **chart** = template-uri YAML + `values.yaml` cu opțiunile expuse. La instalare, Helm randează template-urile cu valorile tale și aplică rezultatul, reținând fiecare **release** cu istoric (de aici rollback). În acest repo nu rulezi `helm install` manual niciodată: Flux are un **helm-controller** care face asta pornind de la obiecte `HelmRelease` din git — primești puterea chart-urilor comunității cu disciplina GitOps.

3. GitOps și Flux: clusterul ca funcție de git

3.1 De ce GitOps și nu „kubectl apply de pe laptop”

Modelul push (CI sau om care aplică manifeste) are trei probleme structurale: nu detectează **drift** (cineva modifică ceva direct în cluster și nimeni nu află), cere credențiale de cluster în CI (suprafață de atac), și nu are o poveste bună de DR. Modelul pull le rezolvă pe toate: un agent **în cluster** trage repo-ul la interval fix și aplică diferențele continuu. Drift-ul e corectat automat

(modificarea manuală e suprascrisă la următoarea reconciliere), nimic extern nu are acces la cluster, iar DR înseamnă „bootstrap Flux pe cluster gol și așteaptă” — exact ce face `task bootstrap:apps`.

3.2 Cele trei controllere Flux

- **source-controller** — clonează surse (GitRepository, HelmRepository, OCIRepository), le verifică și le expune intern ca artefacte. Sursa de adevăr: `meroxdotdev/infrastructure`, branch `main`.
- **kustomize-controller** — aplică obiecte `Kustomization`: ia un director din sursă, îl construiește cu Kustomize și îl aplică prin **Server-Side Apply**. Tot el face decriptarea **SOPS** (cap. 14) și **postBuild substitution** — vezi mai jos.
- **helm-controller** — reconciliază `HelmRelease`: instalează/upgradează chart-uri, cu remediation (retry, rollback) la eșec.

DETALIU CARE CONTEAZĂ: SERVER-SIDE APPLY (SSA)

Flux aplică manifeste cu SSA, în care **fiecare câmp are un „field manager”** (proprietary). Conflictele apar când două entități revendică același câmp — ai văzut asta la DR, unde PVC-urile recreate manual aveau alt field owner decât se aștepta Flux, și scriptul de restore trebuie să corecteze ownership-ul. Când Flux raportează „field is immutable” sau conflicte SSA, întrebarea corectă e „cine mai scrie pe acest obiect?”.

3.3 Structura repo-ului și fluxul de dependențe

```
kubernetes/
├─ flux/cluster/      # Kustomization-ul rădăcină (cluster-apps) – de aici pornește tot
├─ flux/meta/         # HelmRepositories (sursele de chart-uri)
├─ components/common/ # cluster-vars.yaml + componente Kustomize injectate peste tot
└─ apps/<namespace>/<app>/
    ├─ ks.yaml         # Kustomization-ul Flux al aplicației (dependsOn, interval)
    └─ app/             # HelmRelease / manifeste / secret.sops.yaml
```

Două mecanisme țin structura asta scalabilă:

- **dependsOn** pe Kustomization-uri ordonează bootstrap-ul: nimic nu pornește înainte de Cilium (fără CNI podurile nu au rețea), storage înaintea aplicațiilor cu PVC, etc. La un cluster gol, graful ăsta e ceea ce transformă „aplică 40 de aplicații” într-o secvență care converge singură.
- **postBuild substitution**: variabile precum `${LB_IP_GATEWAY_INTERNAL}` sau `${NFS_SERVER}` sunt înlocuite de kustomize-controller la apply, cu valori din `components/common/cluster-vars.yaml`. Asta e singurul loc de editat la mutarea pe hardware nou — IP-urile nu sunt împrăștiate prin 40 de manifeste.

3.4 Renovate: bucla de update-uri

Renovate scanează repo-ul în weekend și deschide PR-uri pentru: versiuni de chart Helm, tag-uri de imagini (adnotate `# renovate:`), versiuni Talos/Kubernetes din `talenv.yaml` / `.mise.toml`. Pattern-ul de reținut: **update-urile devin diff-uri de git reviewabile**, nu comenzi rulate ad-hoc. Tu citești diff-ul, vezi changelog-ul, dai merge — Flux face restul. Config: `.renovaterc.json5`.

PARTEA II

Sistemul de operare: Talos Linux

4. Talos: OS imutabil, gestionat exclusiv prin API

4.1 Problema pe care o rezolvă

Un nod Kubernetes pe un OS clasic (Ubuntu + kubeadm) e un sistem cu două surse de adevăr: starea Kubernetes (declarativă) și starea OS-ului (mutabilă — pachete, config-uri, oameni cu SSH care „repară” lucruri). A doua derivă mereu. Talos elimină complet a doua categorie:

- **Nu există SSH, shell, package manager.** Filesystem-ul rădăcină e read-only, montat dintr-o imagine squashfs semnată. Nu ai cum să faci drift pentru că nu ai unde să-l faci.
- **Singura interfață e un API gRPC** (port 50000, mTLS), vorbit de `talosctl`. „Vreau să văd dmesg” = `talosctl dmesg`. „Vreau procese” = `talosctl processes`. Autentificarea e pe certificate client din `talosconfig` — fără parole, fără chei SSH de rotit.
- **Toată configurația nodului e UN fișier YAML** (machine config): discul de instalare, rețeaua, kernel args, kubelet flags. Aplici config nou → Talos face diff și aplică (unele schimbări cer reboot). Configul devine artefact de git, ca tot restul.
- **Upgrade = înlocuire de imagine**, A/B: noua imagine e scrisă, nodul rebootează în ea; dacă nu boot-ează sănătos, revine automat la cea veche. De aceea `task talos:upgrade-node` e o operație banală, nu un eveniment.

4.2 talhelper și factory images

Machine config-urile nu se scriu de mână: `talos/talconfig.yaml` e o definiție compactă (noduri, IP-uri, discuri, patch-uri) din care **talhelper** generează config-urile complete per nod în `talos/clusterconfig/`, semnate cu secretele clusterului din `talsecret.sops.yaml` (criptat SOPS — cine are fișierul ăsta + `age.key` poate controla clusterul). Imaginile de boot vin de la **factory.talos.dev**: un ID de imagine (hash-ul din `talosImageURL`) codifică exact ce extensii de sistem sunt incluse — aici extensiile pentru **iSCSI** (cerut de Longhorn) și **microcod/GPU Intel** (transcodare Jellyfin). Dacă schimbi extensiile, generezi alt ID pe factory și îl pui în `talconfig`.

4.3 Identitatea nodurilor: MAC → IP static

Fiecare nod e identificat în `talconfig` prin **adresa MAC** (`deviceSelector.hardwareAddr`), cu DHCP dezactivat și IP static (`10.57.57.80/.82/.84`). Asta face providențierea DR elegantă: Terraform creează VM-uri cu MAC-urile de producție, iar nodurile noi primesc automat identitatea (IP, hostname, rol) a celor vechi. Subtilitatea descoperită la testul DR: în **maintenance mode** (înainte de a primi config) Talos folosește totuși DHCP, deci nodurile apar întâi pe IP-uri aleatorii

— taskul `dr:apply-talos-configs` scanează subnet-ul, identifică nodurile după MAC și abia apoi aplică config-ul care le mută pe IP-urile statice.

4.4 VIP-ul de control plane

Kubeconfig-ul arată spre `https://10.57.57.88:6443` — un IP care nu aparține niciunui nod. E un **VIP gestionat de Talos**: nodurile control-plane concurează pe el prin etcd (un fel de leader election), iar câștigătorul își asumă IP-ul pe interfață și răspunde la ARP. Dacă nodul moare, alt nod preia VIP-ul în secunde. Limita de știut: VIP-ul depinde de **etcd funcțional** — dacă pierzi quorumul, pierzi și VIP-ul, deci accesul la API. Pentru situațiile astea, `talosctl` merge direct pe IP-urile nodurilor (`-n 10.57.57.80`), nu prin VIP — ține minte asta când debughezi un cluster „mort”.

ANTI-PATTERN DE EVITAT

Nu trata Talos ca pe un Linux fără shell la care „n-ai încă acces”. Orice nevoie care pare să ceară shell (un fișier pe nod, un `sysctl`, un `mount`) are o formă declarativă în machine config sau e un semn că treaba aia aparține unui pod, nu nodului. Ziua în care „doar de data asta” montezi ceva manual e ziua în care reproducibilitatea moare.

PARTEA III

Rețeaua, cap-coadă

5. Cilium și viața unui pachet în cluster

5.1 Ce e un CNI și de ce Cilium

Kubernetes nu implementează rețea — definește doar contractul: fiecare pod primește un IP unic și orice pod poate vorbi cu orice pod fără NAT. Implementarea o face un plugin **CNI**. Talos e livrat aici cu `cniConfig: none` tocmai ca Cilium să fie singurul CNI — și e instalat la bootstrap înaintea oricărei alte aplicații, pentru că fără CNI niciun pod (nici măcar CoreDNS) nu poate porni.

Cilium e construit pe **eBPF**: programe mici, verificate de kernel, atașate la hook-uri din stiva de rețea, care procesează pachetele în kernel, fără să treacă prin lanțuri iptables. Diferența arhitecturală față de soluția clasică:

Funcție	Clasic (kube-proxy + iptables)	Cilium (eBPF)
Service → pod	Mii de reguli iptables, evaluare liniară per pachet	Lookup O(1) în hash map eBPF la socket/XDP
Network policy	Doar L3/L4, pe IP-uri	Pe identitate (label-uri pod), inclusiv L7
Observabilitate	tcpdump și răbdare	Hubble: flow-uri per pod, verdict policy

De aceea aici `kubeProxyReplacement: true` — kube-proxy nu rulează deloc; Cilium preia integral translatarea Service→endpoint.

5.2 Cum primește un IP extern un serviciu on-prem: L2 announcements

În cloud, un Service de tip LoadBalancer primește IP de la provider. On-prem nu există provider — cineva trebuie să facă două lucruri: să **aloce** un IP (Cilium LB-IPAM, din pool-ul definit în `cilium/app/networks.yaml`) și să-l **anunțe** în LAN. Anunțarea se face prin **L2 announcements**: un nod ales prin leader election răspunde la cererile ARP („cine are 10.57.57.101?") cu MAC-ul propriu, deci traficul LAN pentru acel IP ajunge la el, iar eBPF-ul îl distribuie spre podurile din spate. La căderea nodului, alt nod preia anunțul. E exact modelul MetalLB L2, integrat în CNI. Limită de design: tot traficul unui IP intră printr-un singur nod la un moment dat — irelevant la scara homelab-ului, important de știut la scară mare (acolo s-ar folosi BGP, pe care Cilium îl suportă nativ).

ÎN ACEST SETUP — IP-URILE ANUNȚATE

10.57.57.101 gateway intern (Gateway API) · .102 qBittorrent · .103 Portainer agent · .111 k8s-gateway (DNS) · .112 gateway extern (Cloudflare Tunnel → cluster). Pool-ul și IP-urile sunt variabile postBuild din `cluster-vars.yaml`.

5.3 Gateway API: succesorul lui Ingress

Ingress-ul clasic e un singur obiect supraîncărcat cu adnotări nestandardizate. **Gateway API** îl sparge în roluri: `GatewayClass` (ce implementare — aici Cilium), `Gateway` (listener-ul concret: IP, port, certificat TLS — aici cel intern pe `.101` și cel extern pe `.112`) și `HTTPRoute` (regula per aplicație: „`jellyfin.merox.dev` → service `jellyfin:8096`”, atașată unui `Gateway`). Avantajul operațional: aplicațiile își definesc rutele în propriul namespace fără să poată atinge configurația listener-ului — separare de responsabilitate care la Ingress nu exista.

6. DNS: split-horizon, Pi-hole, Unbound, k8s-gateway

6.1 Principiul split-horizon

Același nume — `jellyfin.merox.dev` — trebuie să răspundă diferit în funcție de cine întreabă: din LAN vrei IP-ul intern (trafic direct, fără ocol prin internet), din exterior vrei ruta Cloudflare Tunnel. Asta e **split-horizon DNS** și e implementat pe trei niveluri:

1. **În cluster:** CoreDNS rezolvă numele de servicii K8s (`svc.cluster.local`) pentru pods.
2. **În LAN:** Pi-hole (pe VPS, accesibil prin Tailscale/LAN) e resolver-ul clienților; pentru zona `merox.dev` face forward către **k8s-gateway** (`10.57.57.111`) — un mic server DNS care rulează în cluster și răspunde cu IP-urile Gateway-urilor/serviciilor pentru fiecare `HTTPRoute` existent. Practic DNS-ul intern se **autogenerează din starea clusterului**: publici o aplicație nouă cu un `HTTPRoute` și numele ei există instant în LAN.
3. **Public:** zona `merox.dev` e la Cloudflare; înregistrările pentru serviciile expuse arată spre tunel (CNAME către `<tunnel-id>.cfargotunnel.com`), gestionate declarativ de `external-dns` (`cloudflare-dns` în namespace-ul `network`).

6.2 Pi-hole + Unbound: de ce două componente

Pi-hole e un **forwarder cu filtrare**: primește query-urile clienților, le blochează pe cele din liste (ad/tracking — răspunde cu IP nul) și le trimite mai departe pe restul. Întrebarea e către cine mai departe. Varianta comodă (8.8.8.8, 1.1.1.1) îți varsă tot istoricul DNS la un terț. De aceea în spatele Pi-hole stă **Unbound**, un **resolver recursiv complet**: pleacă de la root servers (`.`), coboară ierarhia delegărilor (`.dev` → `merox.dev`) și obține răspunsul direct de la serverele autoritative, validând **DNSSEC** pe parcurs. Niciun terț nu vede query-urile agregat; cache-ul Unbound face ca penalty-ul de latență să existe doar la primul query pentru un domeniu.

7. Expunerea spre internet: Cloudflare Tunnel, TLS, cert-manager

7.1 Zero open ports: cum funcționează tunelul

Modelul clasic de self-hosting — port forwarding 80/443 spre un reverse proxy — expune IP-ul tău și o suprafață de atac scanată permanent de tot internetul. Cloudflare Tunnel inversează direcția conexiunii: demonul **cloudflared** (un Deployment în cluster, respectiv integrat pe VPS) deschide **conexiuni outbound** persistente (QUIC/HTTP2) către edge-ul Cloudflare. Când un vizitator cere `jellyfin.merox.dev`, cererea lovește edge-ul Cloudflare, care o împinge înapoi prin tunelul deja deschis către cloudflared, care o livrează serviciului intern (aici: gateway-ul extern Cilium pe `10.57.57.112`).

- **Niciun port inbound deschis** — firewall-ul poate refuza tot traficul nou de intrare. Scanerile găsesc... nimic.
- **IP-ul de origine e invizibil** — DNS-ul public arată doar spre Cloudflare.
- Trade-off-ul de arhitect: Cloudflare **vede traficul în clar** la edge (TLS-ul vizitatorului se termină la ei) și devine dependență critică pentru tot ce e public. Aici e un trade-off acceptat conștient; alternativa (port forwarding + WAF propriu) e mai suverană dar mult mai expusă operațional.

7.2 TLS și ACME: de unde vin certificatele

Un certificat TLS e o legătură semnată criptografic între un nume de domeniu și o cheie publică, emisă de o autoritate (CA) în care browserele au încredere. **ACME** e protocolul prin care Let's Encrypt emite certificate automat, după ce dovedești controlul domeniului printr-o provocare (challenge):

Challenge	Dovada	Limite
HTTP-01	Servești un token pe <code>http://domeniu/.well-known/acme-challenge/</code>	Cere port 80 public; nu emite wildcard
DNS-01	Pui un TXT record <code>_acme-challenge.domeniu</code>	Cere API la DNS; merge fără niciun port deschis și emite wildcard

Cu arhitectura zero-open-ports, **DNS-01 e singura opțiune naturală** — și exact asta fac ambele capete: în cluster, **cert-manager** (un operator: scrii un obiect `Certificate`, el creează TXT-ul prin API-ul Cloudflare, obține certificatul wildcard `*.merox.dev` și îl pune într-un Secret consumat de Gateway, apoi îl reînnoiește singur la ~60 de zile); pe VPS, **Traefik** face același dans ACME DNS-01 și persistă certificatele în `acme.json` (de aceea fișierul ăsta e inclus în backup — pierderea lui nu e gravă, dar reemiterea în masă poate lovi rate limits Let's Encrypt).

8. Tailscale: planul de management

Regula de arhitectură a acestui setup: **traficul public intră prin Cloudflare Tunnel; traficul de management circulă exclusiv prin Tailscale**. Portainer, Homepage intern, kubectl de pe VPS, SSH între mașini — toate pe Tailscale, nimic expus public.

Tailscale e un mesh VPN peste **WireGuard** — un protocol VPN minimal implementat în kernel: fiecare nod are o pereche de chei, pachetele sunt criptate per-peer (Noise/Curve25519,

ChaCha20-Poly1305), fără negociere de cipher-e și fără starea fragilă a IPsec/OpenVPN. Ce adaugă Tailscale peste WireGuard-ul brut e exact partea operațional dureroasă:

- **Distribuția cheilor:** control plane-ul Tailscale (coordination server) distribuie cheile publice și IP-urile (`100.x.y.z`) tuturor nodurilor din tailnet. Tu nu editezi niciodată peers manual. Important: control plane-ul vede doar metadata — **traficul e criptat end-to-end între noduri**, Tailscale nu-l poate citi.
- **NAT traversal:** două mașini în spatele unor NAT-uri diferite nu se pot conecta direct în mod normal. Tailscale folosește STUN ca să descopere maparea porturilor externe și „sparge” majoritatea NAT-urilor pentru conexiuni directe; unde nu reușește (NAT-uri simetrice), retransmite prin relee **DERP** — criptat, releul nu vede conținutul. De aceea „merge pur și simplu” între VPS-ul Oracle și LAN-ul de acasă fără niciun port forward.
- **Identitate, nu IP-uri:** ACL-urile se scriu pe utilizatori/tag-uri. Auth key-ul folosit la provisioning e în Ansible Vault — și e una din dependențele de verificat înainte de DR (expiră!).

9. Traefik: reverse proxy-ul VPS-ului

Pe VPS, echivalentul Gateway-ului din cluster e **Traefik**. Conceptul de reverse proxy: un singur proces deține :80/:443 (aici: primește traficul din tunel), termină TLS-ul și rutează pe baza numelui (SNI/Host header) către serviciul potrivit. Ce îl face potrivit pentru un host Docker e **configurarea prin descoperire**: Traefik ascultă pe socket-ul Docker și își construiește rutele din **label-urile containerelor**. Un serviciu nou = câteva label-uri în compose:

```
labels:
  - traefik.enable=true
  - traefik.http.routers.joplin.rule=Host(`joplin.cloud.merox.dev`)
  - traefik.http.services.joplin.loadbalancer.server.port=22300
```

Nu există fișier central de rute care să fie ținut în sinc — ruta trăiește lângă serviciul ei, în același compose, în git. Middleware-urile Traefik adaugă comportament pe rută; cel mai important aici e **forward-auth către Authentik** (cap. 13), care pune SSO în fața serviciilor care nu știu singure de autentificare. Observație de arhitect asupra simetriei: cluster și VPS rezolvă identic aceeași problemă — un punct de intrare declarativ (Gateway API ↔ labels), TLS automat (cert-manager ↔ ACME built-in), SSO la margine (outpost ↔ forward-auth). Dacă înțelegi una din lumi, o înțelegi și pe cealaltă.

PARTEA IV

Stocare și date

10. Longhorn: block storage distribuit

10.1 Problema stocării on-prem

În cloud, un PVC e satisfăcut de EBS/PD — discuri de rețea pe care providerul le replichează. On-prem ai doar discurile locale ale nodurilor, iar un volum legat de discul unui nod moare odată cu nodul. Longhorn rezolvă asta transformând discurile locale ale tuturor nodurilor într-un **pool de block storage replicat**.

10.2 Arhitectura: un engine per volum, replici sincrone

Designul Longhorn e neobișnuit de simplu pentru storage distribuit: **fiecare volum are propriul său controller** („engine” — un mic proces care rulează pe nodul unde e atașat volumul), conectat la 2-3 **replici** pe noduri diferite. Fiecare scriere e trimisă **sincron** tuturor replicilor — confirmarea vine abia când toate au scris. Consecințe:

- Pierderea unui nod = volumul rămâne disponibil de pe replicile rămase; Longhorn reconstruiește replica lipsă pe alt nod (rebuild = copierea datelor — de aici recomandarea de a aștepta 1-2 ore între înlocuiri de discuri).
- Blast radius minim: un volum corupt nu afectează alte volume — nu există un strat de metadata global care să cadă (spre deosebire de Ceph, care e mai capabil dar mult mai complex operațional; la 3 noduri, Longhorn e alegerea pragmatică).
- Costul: fiecare scriere se face de N ori și traversează rețeaua → throughput mai mic decât discul brut. Pentru config-uri de aplicații e irelevant; pentru fișiere media mari ar fi risipă — de aceea **media stă pe NFS, nu pe Longhorn** (cap. 12).

Volumul e expus podului prin **iSCSI** — motivul pentru care imaginea Talos include extensia iscsi-tools.

10.3 Snapshots vs backups — distincția critică

	Snapshot	Backup
Unde trăiește	Pe aceleași discuri cu volumul (lanț copy-on-write)	În afara clusterului — S3 (aici: Garage pe VPS)
La ce ajută	Revenire rapidă la un punct în timp (upgrade ratat)	Supraviețuiește pierderii întregului cluster
Ce NU acoperă	Moartea clusterului/nodurilor	—

Backup-urile Longhorn sunt **incrementale la nivel de bloc**: primul urcă tot, următoarele doar blocurile schimbate. În S3 ajunge un format propriu Longhorn (nu fișierele tale ca atare) — restaurarea se face tot prin Longhorn, care recrează volumul din blocuri. De aceea procedura DR (cap. 22) are pași expliți de „BackupTarget → sync → restore Volume CRs”.

ÎN ACEST SETUP — OPT-IN, NU OPT-OUT

Backup-ul nightly (02:00, retenție 3) acoperă **doar** volumele etichetate `recurring-job-group.longhorn.io/media: enabled: jellyfin, jellyseerr, prowlarr, qbittorrent, radarr, sonarr` — adică config-urile, singura stare reală. Istoricul Prometheus/Loki/Grafana/Netdata și volumele `*-cache` sunt deliberat excluse (~35GB de zgomot regenerabil). Decizia de design: **backupul e opt-in explicit** — un volum nou NU e backup-uit până nu primește label-ul; prețul e că trebuie să te gândești la asta la fiecare aplicație nouă cu stare reală.

11. Garage: S3 self-hosted ca destinație de backup

S3 nu e un produs, e un protocol — un API HTTP de object storage (PUT/GET pe chei într-un bucket, autentificat cu access/secret key) devenit standardul de facto pentru orice unealtă de backup. Longhorn știe să vorbească S3; tu ai nevoie doar de un server care să-l implementeze. Aici e **Garage** (Deuxfleurs) pe VPS — ales în locul lui MinIO pentru că e mult mai ușor (un singur binar, fără pretenții de cluster enterprise, licență neproblematică) și suficient pentru un singur nod de backup.

Detaliul operațional care contează: Garage ține **datele** (blob-uri pe disc) și **metadatele** (index LMDB — ce obiecte există, în ce bucket). Datele fără metadata sunt iluzibile. De aceea backup-ul Garage-ului însuși (spre NAS) ia întâi un **snapshot consistent de metadata** și abia apoi sincronizează — un rsync naiv peste un LMDB viu poate copia un fișier pe jumătate scris. Restaurarea: copiezi `data/`, restaurezi cel mai recent snapshot ca `meta/db.lmdb`, pornești containerul, iar Longhorn își redescoperă backup-urile.

IMAGINEA DE ANSAMBLU

Lanțul complet: discuri noduri → replici Longhorn → backup incremental → Garage S3 (VPS, off-site față de homelab) → pull nightly pe NAS (off-site față de VPS). Cluster-ul și VPS-ul își sunt reciproc „celălalt loc” — nicio dependență de un cloud de storage plătit.

12. NFS și CSI

Pentru fișierele media (terabytes, citite secvențial, valoarea per-byte mică) replicarea sincronă ar fi risipă. Ele stau pe **NAS-ul Synology** (10.57.57.201) și ajung în pods prin **csi-driver-nfs**:

un StorageClass care montează exporturi NFS direct în containere. NFS-ul e un protocol de file storage în care serverul deține filesystem-ul, iar clienții montează — semantica POSIX e „aproape”, dar file locking-ul peste NFS e istoric fragil; regula practică aplicată și aici: **bazele de date și config-urile aplicațiilor NU pe NFS** (SQLite peste NFS = corupție clasică), doar fișiere mari. De aceea fiecare app *arr are config-ul pe Longhorn și media pe NFS.

PARTEA V

Identitate și secrete

13. Authentik: SSO, OIDC și forward-auth

13.1 De ce un identity provider

Fără SSO, fiecare serviciu are propria bază de utilizatori: N parole, N locuri de dezactivat un cont, N implementări de login de calitate îndoielnică. Un **Identity Provider (IdP)** centralizează: serviciile nu mai verifică parole, ci delegă autentificarea și primesc înapoi o identitate semnată. Aici IdP-ul e **Authentik** (sso.merox.dev, pe VPS, cu PostgreSQL în spate).

13.2 OIDC: cum funcționează delegarea

OpenID Connect (peste OAuth2) — fluxul „authorization code”, pe scurt: aplicația te redirecționează la IdP cu un `client_id`; te autentifici doar la IdP (aplicația nu-ți vede niciodată parola); IdP-ul te redirecționează înapoi cu un cod efemer; aplicația schimbă codul, direct server-to-server, pe un **ID token** — un JWT semnat de IdP conținând identitatea ta (email, grupuri). Aplicația verifică semnătura cu cheia publică a IdP-ului și gata. Răspunsurile la „de unde știe Grafana cine sunt?” sunt toate aici.

Lanțul are două niveluri în acest setup: **Google e upstream IdP pentru Authentik** (social login — Authentik nu stochează parole deloc), iar Authentik e IdP pentru servicii. Politica aplicată: **doar Google login, fără sign-up** — enrollment-ul e închis, conturile noi nu se pot crea; un cont Google necunoscut e respins. Asta mută securitatea parolei pe Google (2FA-ul lor) și reduce Authentik la rol de broker + autorizare.

13.3 Forward-auth și outpost-ul: SSO pentru aplicații care nu știu OIDC

Multe unelte self-hosted nu implementează OIDC. Pentru ele există pattern-ul **forward-auth / proxy auth**: reverse proxy-ul, înainte să livreze orice cerere aplicației, întreabă Authentik „cererea asta are sesiune validă?”. Dacă nu — redirect la login; dacă da — cererea trece, opțional cu identitatea injectată în headere. Aplicația nici nu știe că există autentificare; protecția trăiește în fața ei.

- **Pe VPS**: middleware Traefik de forward-auth către Authentik (ex. Guacamole pe rmt.merox.dev).
- **În cluster**: **authentik outpost** — un mic proxy deployat în K8s, care vorbește cu serverul Authentik de pe VPS și aplică aceleași politici pentru aplicațiile din cluster. Outpost-ul e exact „forward-auth as a deployment”: punctul de decizie rămâne central (VPS), punctul de aplicare e local (cluster).

DEPENDENȚĂ DE ȘTIUT

Cu acest design, **Authentik e single point of failure pentru login** peste tot. De aceea există `make authentik-backup` (de rulat înainte de orice schimbare în Authentik) plus `dump nightly` automat — un Authentik corupt fără backup ar însemna reconfigurarea manuală a fiecărui provider/application/outpost.

14. Secrete: SOPS/age, Ansible Vault, blast radius

14.1 Problema: secretele vor să stea în git, dar nu au voie

GitOps cere ca totul să fie în repo — dar repo-ul conține și token-ul Cloudflare, parola Authentik, cheile S3. Soluția nu e „alt loc pentru secrete” (asta sparge povestea de DR), ci **criptarea secretelor în repo**.

14.2 SOPS + age: cum funcționează

age e o unealtă de criptare asimetrică minimalistă (succesoarea spirituală a GPG, fără complexitatea lui): o pereche cheie publică / cheie privată. **SOPS** e stratul de deasupra care criptează inteligent fișiere YAML: criptează doar **valorile**, nu cheile — un `secret.sops.yaml` rămâne diff-abil în git (vezi că s-a schimbat `api-token`, nu și conținutul). Tehnic e criptare hibridă: SOPS generează o cheie simetrică (data key) per fișier, criptează valorile cu ea (AES256-GCM) și criptează data key-ul cu cheia publică age, stocând rezultatul în metadata fișierului.

Fluxul complet: tu editezi cu `sops secret.sops.yaml` (decriptează în editor, recriptează la save) folosind cheia privată din `age.key`; commit; **kustomize-controller** din Flux, care are `age.key` într-un Secret (`sops-age`, creat la bootstrap), decriptează la apply și scrie obiectul Secret real în cluster. Nimic necriptat nu atinge git-ul vreodată.

AGE.KEY ESTE CLUSTERUL

Cine are repo-ul (public!) + `age.key` poate citi toate secretele și, prin `talsecret.sops.yaml`, poate controla clusterul. Invers: dacă **pierzi** `age.key`, niciun secret din repo nu mai poate fi decriptat vreodată — DR-ul devine imposibil în forma documentată. De aceea e singurul artefact cu cerință absolută de copii off-site, ținute separat de repo. Rotirea: generezi cheie nouă → `find . -name "*.sops.*" -exec sops updatekeys {} \;` → înlocuiești secretul din cluster.

14.3 Ansible Vault și inventarul complet al secretelor

Pe partea de VPS, echivalentul e **Ansible Vault**: `vault.yml` criptat AES256 cu o parolă (în `.vault_pass`, citit automat de `ansible.cfg`). Diferența de model față de SOPS: Vault criptează fișierul întreg (diff-uri opace) și e simetric (o parolă, nu pereche de chei) — mai primitiv, dar nativ în Ansible. Harta completă a secretelor, cu ce NU e recuperabil din git:

Secret	Unde stă	Recuperabil din git?
Secrete K8s (Cloudflare, Authentik, S3 Longhorn)	*.sops.yaml în repo	Da, cu age.key
age.key	infrastructure/age.key (gitignored)	NU — copie off-site obligatorie
Secrete VPS (Tailscale, Cloudflare, Garage)	Ansible Vault în repo	Da, cu .vault_pass
.vault_pass	vps/.vault_pass (gitignored)	NU — copie off-site
Parole Docker (Pi-hole, Joplin DB...)	/srv/docker/oracle-cloud/.env	NU — copie off-site
Telegram, Tavily, Anthropic	~openclaw/.openclaw/.env	NU — copie off-site
Secrete bootstrap Talos	talos/talsecret.sops.yaml (SOPS)	Da, cu age.key

14.4 Gândirea în blast radius

Întrebarea de arhitect nu e „sunt în siguranță?” ci **„dacă mașina X e compromisă, ce cade?”**. Aplicată aici: VPS-ul e expus internetului (prin tunel) și ține age.key, kubeconfig și talosconfig — deci un VPS compromis înseamnă, tranzitiv, cluster compromis. Atenuările aplicate (hardening 2026-06-11) reduc suprafața, iar decizia rămasă deschisă e mutarea age.key off-VPS — trade-off direct între securitate (lanțul se rupe) și confortul operațional (sops/talhelper rulate de pe VPS). Principiul general de reținut: **credențialele de management pe mașina cea mai expusă sunt cea mai scumpă comoditate**; inventariază-le periodic și mută ce nu e folosit zilnic.

PARTEA VI

Observabilitate

15. Prometheus, Grafana, Alertmanager

15.1 Modelul pull și de ce contează

Prometheus inversează intuiția: aplicațiile **nu trimit** metrici nicăieri — expun un endpoint HTTP (`/metrics`, text simplu: nume, label-uri, valoare), iar Prometheus le **colectează (scrape)** la interval fix. Avantajele care au făcut modelul standard: ținta nu are nevoie de config (nu știe unde e serverul de monitoring); Prometheus știe imediat când o țintă e moartă (scrape failed = metrica `up == 0` — monitorizarea sănătății e gratuită); și descoperirea țintelor e dinamică — în Kubernetes, prin **ServiceMonitor/PodMonitor** (CRD-uri ale Prometheus Operator): orice chart care livrează un ServiceMonitor e monitorizat automat, fără să atingi configul Prometheus.

Stack-ul vine din chart-ul **kube-prometheus-stack**: Prometheus Operator + Prometheus + Alertmanager + node-exporter (metrici de host per nod) + kube-state-metrics (starea obiectelor K8s ca metrici) + reguli de alertare predefinite. Datele stau într-un **TSDB** local pe Longhorn — compresie bună, dar deliberat **fără backup**: istoricul de metrici e regenerabil prin definiție (se reumple singur), iar în DR Prometheus pornește gol.

15.2 PromQL minim pentru operare

```
up == 0                                # ținte moarte (primul query în orice incident)
rate(container_cpu_usage_seconds_total[5m])  # CPU per container — counters se citesc cu
rate()                                       #
container_memory_working_set_bytes          # memoria "reală" (asta vede OOM killer-ul)
kubelet_volume_stats_available_bytes         # spațiu liber pe PVC-uri — alerta de disc plin
```

Regula de aur: **counters** (totaluri monoton crescătoare) se interoghează întotdeauna prin `rate()`; **gauges** (valori instantanee) se citesc direct.

15.3 Alertmanager și dead-man's switch-ul

Separația de responsabilitate: Prometheus **evaluatează** regulile de alertare (expresie PromQL adevărată pe o durată → alertă firing), Alertmanager **livrează** — grupează (10 poduri căzute = o notificare, nu 10), face deduplicare și rutează. Cea mai subtilă piesă: alerta **Watchdog** — o alertă care e mereu firing, trimisă continuu către **healthchecks.io** ca heartbeat. Dacă heartbeat-ul se oprește, healthchecks.io (serviciu extern!) te anunță. Asta răspunde la întrebarea pe care monitoringul naiv o ignoră: cine monitorizează monitoringul? Un Prometheus

mort nu poate raporta că e mort — semnalul lipsei de semnal trebuie să ajungă în afara sistemului.

15.4 Grafana

Grafana e stratul de vizualizare peste surse multiple (Prometheus + Loki aici). Detaliul de arhitectură GitOps: dashboard-urile **nu se salvează din UI** — sunt JSON-uri în git, încărcate de un sidecar de provisioning care urmărește ConfigMaps. De aceea în DR Grafana pornește cu toate dashboard-urile la loc deși volumul ei e nou: UI-ul e doar o proiecție a git-ului. Login: prin Authentik (OIDC).

16. Loguri: Loki și Promtail

Loki aplică logurilor filozofia Prometheus: **indexează doar label-urile** (namespace, pod, container), nu și conținutul liniilor — spre deosebire de Elasticsearch, care indexează full-text fiecare cuvânt. Rezultatul: cost de stocare și RAM cu un ordin de mărime mai mic, în schimbul căutării full-text mai lente (grep peste chunk-urile comprimate ale stream-ului selectat). Pentru un homelab, exact trade-off-ul corect — întrebarea reală e mereu „logurile podului X din ultimele 2 ore”, nu „caută «error» în tot ce există”.

Promtail e colectorul: DaemonSet pe fiecare nod, citește fișierele de log ale containerelor de pe nod (kubelet le scrie în `/var/log/pods/`), atașează label-urile K8s și împinge în Loki. Interogarea — LogQL, în Grafana (sintaxa: selector de stream + filtru de text):

```
{namespace="default", pod=~"jellyfin.*"} |= "error"      # liniile cu "error" din podurile jellyfin
```

Ca și metricile, logurile sunt **deliberat fără backup** — istorie regenerabilă, nu stare.

17. Restul flotei: Netdata, Uptime Kuma, Beszel, Dozzle, Glances

Pare redundantă — sunt cinci unelte de „monitoring” — dar fiecare răspunde la altă întrebare, și e instructiv să le vezi ca straturi:

Unealtă	Întrebarea la care răspunde	Model
Prometheus/Grafana	„Ce tendințe și alerte am în cluster?”	Pull, TSDB, sursa de adevăr istorică
Netdata	„Ce se întâmplă pe host chiar acum?” — granularitate 1s, mii de metrici auto-descoperite	Parent pe VPS + 3 child nodes; debugging live, nu istoric
Uptime Kuma (status.merox.dev)	„Serviciile răspund din perspectiva utilizatorului?” — probe HTTP/TCP/ping sintetice + alerting	Black-box: testează ce vede lumea, nu ce simte serverul
Beszel	„CPU/RAM/disc pe toate gazdele, dintr-o privire?”	Hub + agenți minusculi; complementul ușor al Netdata
Dozzle	„Logurile containerelor Docker de pe VPS, live?”	UI peste Docker API — echivalentul VPS al Loki (fără stocare)
Glances	„htop în browser pentru VPS”	Vedere instant per-proces

Distincția conceptuală de reținut: **white-box** (metrici interne — Prometheus, Netdata) îți spune de ce e stricat; **black-box** (probe externe — Uptime Kuma) îți spune că e stricat, din perspectiva care contează. Ai nevoie de ambele; alerting-ul primar pe „e jos?” e treaba black-box-ului.

PARTEA VII

VPS-ul: Ansible, Terraform și serviciile Docker

18. Ansible și Terraform: VPS-ul ca artefact reproducibil

18.1 Diviziunea muncii dintre cele două

Confuzia clasică „Terraform sau Ansible?” are un răspuns curat aici, pentru că fac lucruri ortogonale: **Terraform creează mașini** (resurse la un cloud provider, cu un fișier de stare care ține evidența a ce există ca să poată calcula diff-ul și să distrugă curat), **Ansible configurează mașini** (rulează pași idempotenți prin SSH, fără agent și fără stare — starea e mașina însăși). În acest repo: Terraform e folosit doar pentru DR (provisionare Hetzner la pierderea Oracle-ului; și pentru VM-urile DR Proxmox pe partea de cluster), Ansible pentru tot ce înseamnă conținutul VPS-ului.

18.2 Idempotența — proprietatea care definește totul

Un playbook Ansible bun nu e un script de instalare, e o **declarație de stare**: „pachetul X prezent, fișierul Y cu acest conținut, containerul Z pornit”. Rulat de 100 de ori, schimbă ceva doar dacă realitatea diferă — de aceea `make setup` e și instalare inițială, și „repară tot ce a deviat”, și deploy de schimbări, aceeași comandă. Corolarul disciplinar: **orice modificare manuală pe VPS în afara Ansible va fi suprascrisă sau, mai rău, uitată** — exact problema pe care Talos o rezolvă prin imposibilitate, Ansible o rezolvă prin disciplină. (Gap cunoscut și asumat: `filebrowser` și `nas_mount` rulează pe VPS fără rol Ansible — nu ar supraviețui unui `make dr-full`.)

Structura: **un rol per serviciu** (`traefik_setup`, `authentik_setup`, `pihole_setup` ... ~19 roluri), fiecare deținând template-urile `compose`, `config`-urile și `task`-urile lui; inventarul de producție cu particularitatea `ansible_connection=local` — Ansible rulează pe VPS însuși (OCI blochează SSH inbound de pe IP-uri arbitrare), în timp ce inventarul DR merge clasic peste SSH.

18.3 make dr-full: anatomia unui failover de 15 minute

1. Pre-flight: vault deschis, token Hetzner, cheie SSH prezente.
2. Terraform creează VPS-ul Hetzner, scrie IP-ul nou în inventar (și scoate `ansible_connection=local`).
3. Poll SSH până mașina răspunde, apoi `make setup` — toate cele ~19 roluri.

4. Reconectarea „magică” e de fapt design: **Cloudflare Tunnel, Tailscale și Let's Encrypt au credențiale în vault**, deci serviciile de pe mașina nouă se înregistrează singure la aceleași identități — DNS-ul public nu se schimbă (arăta spre tunel, nu spre IP), tailnet-ul primește nodul înapoi, certificatele se reemit prin DNS-01.
5. Rămâne datele: `make restore` (wizard interactiv pentru dump-urile Authentik/Joplin de pe NAS) + untar state-ul de servicii + copierea Garage.

Lecția generalizabilă: failover-ul rapid nu vine din scripturi rapide, ci din **eliminarea identităților legate de mașină** (IP-uri publice, certificate locale, stare nereproductibilă) din design, cu mult înainte de dezastru.

19. Serviciile de pe VPS, unul câte unul

Toate sub Docker Compose în `/srv/docker/oracle-cloud/`, toate în spatele Traefik + Cloudflare Tunnel, management prin Tailscale. Traefik (cap. 9), Pi-hole+Unbound (cap. 6), Authentik (cap. 13), Garage (cap. 11) și flota de monitoring (cap. 17) au fost tratate la capitolele lor; restul:

Serviciu	Ce e și ce rol joacă aici
Joplin Server joplin.cloud.merox.dev	Sincronizare de notițe end-to-end (clienții Joplin). Stare reală: PostgreSQL → <code>pg_dump</code> nightly 01:20, pe NAS la 03:30. Una din puținele aplicații de aici cu date cu adevărat de neînlocuit.
Guacamole rmt.merox.dev	Gateway de remote desktop clientless: RDP/VNC/SSH randate în browser (HTML5) — serverul Guacamole vorbește protocoalele native cu țintele, browserul vede doar un canvas. În spatele forward-auth Authentik: accesul la toate desktop-urile = un singur login SSO. Conexiunile salvate intră în backup-ul de extras (01:30).
Portainer EE doar Tailscale, :9000	UI de management Docker multi-host; vede clusterul prin portainer-agent (pod în K8s pe LB <code>10.57.57.103</code>). Util pentru inspecție rapidă; sursa de adevăr rămâne git-ul — orice schimbare făcută din Portainer pe resurse gestionate de Flux/Ansible e drift care va fi suprascris.
Homepage inside.merox.dev (Tailscale)	Dashboard intern cu widget-uri live (K8s, Proxmox, router) — config declarativ, există și instanță în cluster. Pagina de start a operatorului.
Uptime Kuma status.merox.dev	Probe sintetice + pagina de status publică (cap. 17). Istoricul e deliberat ne-backup-uit.
Code Server code.cloud.merox.dev	VS Code în browser, pe VPS — editare de oriunde, cu repo-urile local. Parola în <code>.env</code> .
Filebrowser	UI de fișiere peste directoarele serverului. Atenție: fără rol Ansible încă (gap DR documentat).
agents-dashboard agents.cloud.merox.dev	nginx static peste <code>/srv/dashboard</code> — command center-ul agenților OpenClaw (cap. 23), actualizat de agenți + cron.

PATTERN-UL COMUN

Fiecare serviciu = un director cu `docker-compose.yml` (din template Ansible) + label-uri Traefik + secrete din `.env` + IP static `172.25.10.x`. Un serviciu nou se adaugă scriind un rol nou după același tipar — nu instalând manual și sperând că ții minte cum.

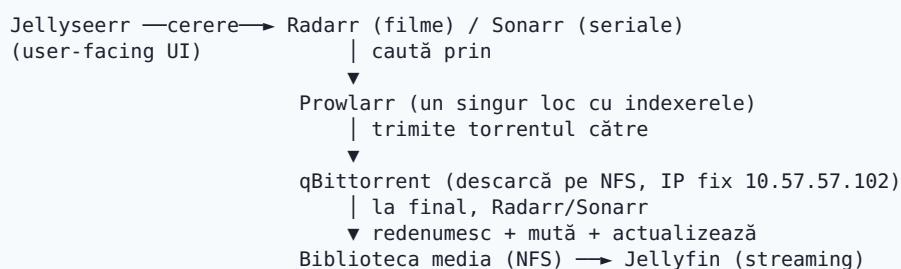
PARTEA VIII

Aplicațiile din cluster

20. Media stack-ul și restul aplicațiilor

20.1 Pipeline-ul media: cum colaborează *arr-urile

Stack-ul media e un pipeline cu responsabilități stricte — fiecare componentă face un singur lucru și vorbește cu vecinii prin API:



- **Jellyseerr** — interfața de „cereri”: utilizatorii caută un titlu, el îl trimite la Radarr/Sonarr. Singura componentă pe care o ating non-adminii.
- **Radarr / Sonarr** — manageri de colecție pentru filme / seriale: decid ce calitate, monitorizează apariții noi, organizează fișierele după convenții. Creierul automatizării.
- **Prowlarr** — agregatorul de indexere: definești sursele o singură dată, el le sincronizează în Radarr/Sonarr. Fără el, fiecare app și-ar gestiona propriile indexere.
- **qBittorrent** — clientul de download, pe LoadBalancer cu IP fix (port forwarding și acces UI stabil).
- **Jellyfin** — serverul de streaming, echivalentul self-hosted al Plex fără componenta cloud.

Toate respectă același tipar de stocare (cap. 10/12): **config pe Longhorn** (mic, replicat, backup-uit nightly prin label-ul de recurring job) + **media pe NFS** (mare, ne-replicat, re-descărcabil). Asta face întreg stack-ul restaurabil în DR din câteva volume mici.

20.2 Transcodarea hardware: GPU-ul în Kubernetes

Transcodarea video pe CPU e prohibitivă (un stream 4K → toate core-urile). GPU-urile Intel au **Quick Sync** — encodere dedicate care fac asta aproape gratuit. Ca un pod să primească GPU: (1) imaginea Talos include driverele i915 (extensie factory); (2) **intel-device-plugin-operator** publică GPU-ul ca resursă alocabilă (gpu.intel.com/i915); (3) HelmRelease-ul Jellyfin cere resursa, deci scheduler-ul îl pune doar pe noduri cu GPU (label-ul `intel.feature.node.kubernetes.io/gpu` de pe nodul 1 — Beelink-ul). Consecința DR cunoscută:

pe VM-uri DR fără GPU, Jellyfin rămâne Pending până scoți cererea de resursă — software transcoding sau direct play, dar funcțional.

20.3 Restul aplicațiilor și utilitarele de cluster

Componentă	Rol
n8n	Automatizare de workflow-uri (node-based, self-hosted Zapier). Stare în PVC propriu.
metrics-server	Alimentează <code>kubectl top</code> și HPA cu metrice instant de CPU/RAM. (Diferit de Prometheus — el e doar pipeline-ul intern de resurse.)
reloader	Repornește automat podurile când li se schimbă ConfigMap/Secret-ul — Kubernetes nu face asta nativ; fără el, schimbările de config „nu se aplică” până la un restart manual.
spegel	Mirror de registry P2P între noduri: imaginile deja trase de un nod se servesc local celorlalte. Pull-uri mai rapide și imunitate la rate-limit-uri/indisponibilitatea registry-urilor (important la un cold start de cluster, când totul trage imagini deodată).
csi-driver-nfs	StorageClass-ul NFS (cap. 12).
coredns	DNS-ul intern al clusterului (cap. 6).
restore-pvs	Manifestele/logica de re-legare a volumelor restaurate din S3 (cap. 22).

PARTEA IX

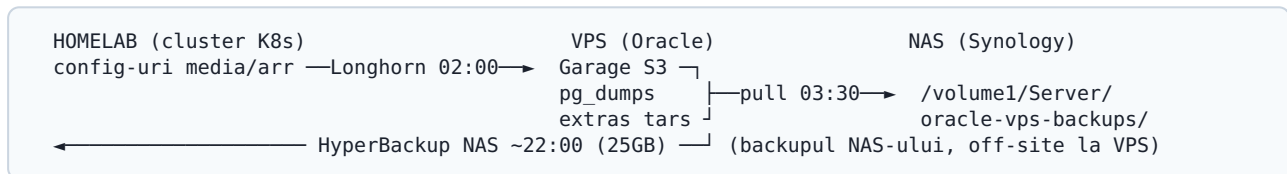
Backup și Disaster Recovery

21. Strategia de backup: doar stare reală, off-site reciproc

21.1 Principiul: nu face backup la ce poți reconstrui

Punctul de plecare al oricărei strategii sănătoase: împarte fiecare byte în **declarativ** (reconstruibil din git — manifeste, config-uri Ansible, secrete criptate), **regenerabil** (cache-uri, istoric de metrice/loguri — se refac singure) și **stare reală** (baze de date, config-uri acumulate prin UI — pierdute = pierdute). Backup-ul acoperă **doar a treia categorie**. Aici asta a redus volumul de la ~35GB de zgomot la câteva sute de MB de date care contează — iar un backup mic e un backup pe care chiar îl testezi.

21.2 Topologia: reciproc off-site



Fiecare locație e „off-site-ul” alteia — fără cloud de storage plătit: clusterul își ține backup-urile pe VPS (Garage S3), VPS-ul își ține totul (inclusiv Garage!) pe NAS, NAS-ul își ține propriul backup pe VPS. Singura grijă topologică: `/backup/synology` (HyperBackup-ul NAS-ului pe VPS) e **exclus** din sincronizarea spre NAS — altfel ai sincroniza backupul NAS-ului înapoi pe NAS, circular.

21.3 Orarul nopții, complet

Ora	Ce rulează	Detalii
~22:00	NAS → VPS: HyperBackup	~25GB, backupul propriu al NAS-ului, în <code>/backup/synology</code>
01:15 / 01:20	pg_dump Authentik / Joplin	→ <code>/srv/backups/</code> , retenție 7 zile (rolurile <code>authentik_setup / joplin_setup</code>)
01:30	<code>backup-vps-extras.sh</code>	Tars: conexiuni Guacamole, <code>acme.json</code> Traefik, config Pi-hole (fără <code>history/gravity</code>), runtime OpenClaw (fără <code>.env/logs</code>). Retenție 7 zile.
02:00	Longhorn → Garage S3	Volumele cu label-ul media (configs <code>*arr/jellyfin</code>), retenție 3
03:30	<code>backup-push-nas.sh</code>	Snapshot consistent de metadata Garage, apoi VPS-ul declanșează prin SSH NAS-ul, care trage <code>/srv/backups/</code> + datele Garage din modulele <code>rsyncd read-only</code> ale VPS-ului

DE CE PULL ȘI NU PUSH — O LECȚIE DESPRE REALITATE VS TEORIE

Planul natural era push: VPS-ul împinge cu `rsync` peste SSH pe NAS. Dar DSM-ul Synology livrează un `rsync` patch-uit care refuză modul server peste SSH dacă „rsync service” nu e activat din UI. În loc de a lupta cu DSM-ul, soluția folosește ce funcționează: NAS-ul ca **client** `rsync`, trăgând din demonul `rsyncd` pe care VPS-ul îl rulează oricum pentru HyperBackup. Modulele `rsyncd` sunt `read-only` — bonus de securitate: NAS-ul compromis nu poate scrie pe VPS. Generalizarea: pe appliance-uri (DSM, QNAP, routere) verifică întotdeauna ce e realmente disponibil înainte să proiectezi; și preferă pull dinspre sistemul care stochează — direcția potrivită și pentru ransomware resilience.

21.4 Ce rămâne manual — și de ce e OK doar dacă e scris

Patru fișiere nu pot fi în niciun backup automat de pe VPS, pentru că ele sunt cheile backupului însuși sau secrete în clar: `age.key`, `vps/.vault_pass`, `~openclaw/.openclaw/.env`, `/srv/docker/oracle-cloud/.env`. Acestea cer copii **off-VPS ținute de om** (password manager / disc offline). Plus o singură provisionare manuală pe NAS (cheia SSH a VPS-ului la `admin@NAS`, fișierul de parolă `rsync`) — documentată în README-ul rolului, pentru că „manual și nedocumentat” e cum mor DR-urile.

22. Scenarii DR și proceduri de restaurare

22.1 Harta scenariilor

Scenariu	Procedură	Durată	RPO (date pierdute, max)
Cluster K8s mort	DR.md : VM-uri DR → bootstrap → restore S3	~35 min (testat 2026-06-06)	24h (backup nightly)
VPS pierdut (Oracle reclamă free tier)	make dr-full → make restore + Garage de pe NAS	~15 min + restore	24h (sync nightly pe NAS)
Tot, de la zero	DEPLOY.md : Faza 1 (VPS) → 2 (K8s) → 3 (agenți)	~50 min	24h
O bază de date stricată (Authentik/Joplin)	make restore — wizard, alegi dump-ul	minute	≤24h

22.2 Restaurarea clusterului, pe faze (esența DR.md)

- Provision:** `task dr:create-vms` — Terraform creează 3 VM-uri pe Proxmox cu MAC-urile de producție; `task dr:apply-talos-configs` scanează subnetul, identifică nodurile după MAC și le aplică config-urile → reboot pe IP-urile statice .80/.82/.84.
- Bootstrap:** `talhelper gencommand bootstrap` (etcd), apoi kubeconfig. Nodurile sunt NotReady — normal, nu există încă CNI.
- Apps:** `task bootstrap:apps` — instalează Flux, care instalează tot restul din git în ordinea dependsOn (Cilium → storage → apps). Aici se vede payoff-ul GitOps: nu „restaurezi” aplicațiile, le re-derivezi.
- Date:** `task longhorn:restore` — secvența automatizată: BackupTarget → S3; așteaptă sincronizarea CR-urilor BackupVolume și Backup (cele din urmă întârzie ~30s — primul bug găsit la test); creează volumele de restore; aplică PV-uri cu claimRef corect; curăță claimRef-uri stale și field ownership SSA; PVC-uri proaspete pentru observabilitate (deliberat goală); force-reconciliază HelmRelease-urile.
- Verify:** poduri Running, PVC-uri Bound, HelmRelease-uri Ready. Excepții așteptate: Jellyfin Pending fără GPU; monitoringul pornește gol; Jellyfin cere câțiva pași manuali de UI post-restore (rețele LAN, proxy — documentați în `docs/jellyfin-post-restore.md`).

CEA MAI IMPORTANTĂ PROPOZIȚIE DIN ACEST CAPITOL

Un backup netestat nu există, iar un DR netestat e o speranță. Procedura de mai sus are valoarea pe care o are pentru că a fost rulată cap-coadă (2026-06-06) și fiecare surpriză găsită (Backup CR-uri întârziate, claimRef stale, DHCP în maintenance mode, label-ul de discuri Longhorn) a devenit fix în repo. Re-testează după orice schimbare structurală de storage sau rețea.

PARTEA X

Agenții AI și operarea de zi cu zi

23. OpenClaw: agenți AI peste infrastructură

Stratul final: un **gateway OpenClaw** (serviciu systemd user, sub userul dedicat `openclaw`) care leagă un bot Telegram de Claude și de unelte locale. Arhitectural, un agent = **model + unelte + spațiu de lucru**: primește mesajul, decide ce comenzi să ruleze (kubectl, docker — definite ca „skill”-uri, ex. `agent/skills/infra/`), execută și răspunde. Agenții configurați: **news** (briefing zilnic), **blog** (publicare pe merox.dev), **infra** (comenzi pe cluster/VPS), **costs**, **design**, **orchestrator** (rutare + task-uri programate), **dashboard** (actualizează `agents.cloud.merox.dev`), **renovate**.

Deciziile de design care merită reținute ca pattern de „AI cu acces la infrastructură”:

- **Identitate separată**: userul `openclaw` are kubeconfig/talosconfig proprii — accesul agenților e enumerabil și revocabil, nu „root pe tot”.
- **Config în git, secrete în afara lui**: `agent/openclaw.json` (template, în repo) vs `~/.openclaw/.env` (Telegram token, chei API — gitignored, copie manuală off-site).
- **Runtime-ul nu e stare prețioasă**: workspace-urile și logurile agenților intră în tarul de extras fără `.env` /loguri; reinstalarea completă e Faza 3 din DEPLOY.md (~15 min).

24. Day-2 operations: comenzi, tipare de debugging, upgrade-uri

24.1 Tiparul universal de debugging în Kubernetes

Aproape orice incident de cluster se diagnostichează coborând aceeași scară — de la intenție spre execuție:

1. **Flux a aplicat ce e în git?** — `flux get kustomizations -A`, `kubectl get helmreleases -A | grep -v True`. Dacă nu: `flux logs --level=error`, apoi reconciliere forțată sau suspend/resume pe HelmRelease-ul blocat.
2. **Obiectele există și ce zice controllerul?** — `kubectl describe` pe resursa custom; secțiunile `status` și `Events` sunt locul unde operatorii își explică refuzurile.
3. **Podul rulează și de ce nu?** — `describe pod` (scheduling? imagine? probe-uri?), `logs -f` și mai ales `logs --previous` la crash loop-uri (logurile dinaintea restartului conțin cauza).
4. **Nodul e sănătos?** — `talosctl -n <ip> health / dmesg / services`; ține minte: direct pe IP de nod, nu prin VIP, dacă API-ul e jos.

24.2 Cheat sheet operațional

```
# Cluster – puls
kubectl get nodes && kubectl get pods -A | grep -v Running
task reconcile # forțează sync Flux după un push

# Talos – ciclul de viață
task talos:generate-config # după editarea talconfig.yaml
task talos:apply-node IP=10.57.57.80
task talos:upgrade-node IP=... # A/B, revine singur dacă eșuează
task talos:upgrade-k8s

# VPS
cd vps/
make health-check # verificarea tuturor serviciilor
make check # dry-run (--check --diff) ÎNAINTE de setup
make setup # deploy idempotent complet
make backup-sync-now # backup extras + sync NAS, acum

# Backup – verificare că trăiește
kubectl get backupvolumes.longhorn.io -n longhorn-system # ultimul backup per volum
docker exec garage /garage status
```

24.3 Upgrade-uri fără frică

Ordinea de încredere: (1) **Renovate** deschide PR-ul — citești changelog-ul, te uiți la major versions; (2) merge → Flux aplică; (3) dacă doare → `git revert` și Flux face rollback. Pentru Talos/K8s: noduri pe rând (`upgrade-node` per IP, apoi `upgrade-k8s`), clusterul rămâne sus pentru că etcd tolerează un nod lipsă. Pentru schimbări de disc fizic: drain → swap → apply config → uncordon, și **1-2 ore între noduri** pentru rebuild-ul replicilor Longhorn — graba la al doilea disc înainte ca primul să-și refacă replicile e singurul mod ușor de a pierde date cu replicare 2/3.

25. Glosar

Termen	Definiție scurtă
ACME / DNS-01	Protocolul de emitere automată a certificatelor TLS; varianta de challenge prin TXT record — merge fără porturi deschise, permite wildcard.
age / SOPS	Criptare asimetrică minimalistă / unealta care criptează valorile din YAML cu ea — secrete versionabile în git.
CNI / CSI / CRI	Interfețele standard prin care Kubernetes delegă rețeaua (Cilium), stocarea (Longhorn, NFS) și runtime-ul de containere (containerd).
CRD / operator	Tip nou de obiect în API-ul K8s / controllerul care îl reconciliază — mecanismul de extensie al întregului ecosistem.
eBPF	Programe verificate rulate în kernel — baza Cilium pentru rutare, load-balancing și policy fără iptables.
etcd / Raft / quorum	Baza de date a clusterului / algoritmul ei de consens / majoritatea necesară scrierilor (2 din 3) — motivul numărului impar de control-plane-uri.
forward-auth / outpost	Pattern-ul în care proxy-ul validează sesiunea la IdP înaintea fiecărei cereri / agentul Authentik care îl aplică în cluster.
GitOps / reconciliere	Starea dorită în git, agenți în cluster care o aplică continuu; drift-ul e corectat, nu doar detectat.
HelmRelease / Kustomization	CRD-urile Flux: un chart Helm gestionat declarativ / un director de manifeste aplicat din sursă, cu dependsOn și postBuild.
L2 announcement / VIP	Anunțarea unui IP prin ARP de către un nod ales — cum primesc IP-uri externe serviciile on-prem și cum „plutește” API-ul pe 10.57.57.88.
PVC / PV / StorageClass	Cererea de stocare a aplicației / volumul concret / rețeta de provisionare — contractul care decuplează aplicația de infrastructura de storage.
RPO / RTO	Câte date poți pierde (aici: $\leq 24h$) / cât durează revenirea (aici: 15–50 min) — cele două numere care definesc orice strategie de DR.
Server-Side Apply	Modul de aplicare K8s cu proprietate per câmp ("field manager") — sursa conflictelor când două sisteme scriu pe același obiect.
Split-horizon DNS	Același nume, răspunsuri diferite intern vs extern — LAN-ul merge direct, internetul prin tunel.
WireGuard / DERP	Protocolul VPN minimal din kernel pe care e construit Tailscale / releele criptate folosite când NAT traversal-ul direct eșuează.
Zero open ports	Postura în care nimic nu ascultă inbound public: expunerea prin conexiuni outbound (Cloudflare Tunnel), managementul prin mesh VPN (Tailscale).